School of Computer and Information Technology

Beaconhouse National University

Software Engineering

Class Activity

13-November-2025

Activity 1 : Classification of Requirements

Read the case study carefully. On a separate sheet of paper:

1. **Identify and Classify all the requirements mentioned in the case study in to one of the following categories:**
    - Functional Requirement
    - Non-Functional Requirement
    - Domain Requirement
    - Inverse Requirement
    - Design and Implementation Constraint

Beaconhouse National University (BNU), Lahore, plans to introduce a digital Campus Management Solution (CMS) called *BNU CampusConnect* to enhance operational efficiency and improve student experience. The system should allow students to register for courses, view timetables, check attendance records, and submit assignments online. Faculty members should be able to upload grades, share lecture materials, and communicate with students through the portal. The administration requires the system to manage student admissions, fee payments, and transcript generation in a centralized manner. Students should also be able to access notifications about upcoming events, academic deadlines, and financial dues through the web portal and mobile app.

The university has specified that the system must ensure data security and privacy, especially concerning student records and financial information. It must comply with the Higher Education Commission (HEC) data management guidelines for educational institutions. The platform should handle at least 10,000 concurrent users during peak registration periods, maintain system uptime of 99.9%, and support both English and Urdu interfaces for inclusivity. The user interface should be accessible to users with limited technical skills and optimized for mobile devices. Furthermore, the CMS should integrate seamlessly with Microsoft Active Directory for single sign-on (SSO) access across all university systems.

BNU's academic policies introduce certain domain-specific requirements. The system must automatically calculate CGPA and academic standing as per BNU's grading policy. It should restrict students from enrolling in a course if prerequisite courses have not been completed, and generate attendance shortage warnings automatically before the end of the semester. The solution should also support elective course selection workflows for various departments according to their unique program structures. Additionally, the system must produce customized reports for accreditation and quality assurance as required by HEC and internal audit committees.

The university has also defined some design and implementation constraints. The system must be developed using .NET Core for the backend and Angular for the frontend, while the database should use Microsoft SQL Server to align with existing IT infrastructure. The CMS should run on the university's on-premise servers instead of a cloud platform for data control and cost reasons. The system must not allow students to alter attendance records, modify grades, or delete submitted assignments, which are treated as inverse requirements to prevent misuse. Finally, the entire solution must be delivered within eight months and remain within a budget of PKR 15 million.

Activity 2 : Finding Errors in Requirements and Rewriting the Requirements

Listed below are 10 requirements of bespoke software that is being developed for BNU. Perform the following tasks

- Identify the error (s) in the requirement
- Rewrite the requirement in correct format

During this activity you should:

- focus on identifying common errors in requirements.
- look for requirement quality attributes which are correctness, clarity, completeness, feasibility, and verifiability.
- try to convert "soft" or vague statements into "hard" or measurable requirements and practice writing high-quality requirements that are clear, measurable, testable, and feasible
- have valid justification of the rewritten requirement and how it resolves the identified errors. You don't have to mention the justification, you just need to think logically and solve on the basis of a valid reasoning.

1. The app must never crash.
2. The interface should look like Facebook.
3. The user should be able to login easily.
4. The system should be highly secure.
5. The app should use modern technologies.
6. The system must never lose user data.
7. The system should send an email whenever needed.
8. The UI should be attractive and beautiful.
9. The application should have many useful features.
10. The server should always be available.

Activity 3 : Choosing the Right process Model

Given below are different scenarios of software projects. Based on the information given, which model will you use for developing these system and why? A valid justification is required in the answer. No marks will be awarded if only the process model is mentioned.

Scenario 1 – Personalized Fitness App

A startup wants a personalized fitness and diet tracking app. The initial idea is broad, with unclear requirements regarding AI-driven meal suggestions and workout routines. They want a prototype quickly to show potential investors and get user feedback. Changes are expected as user preferences and AI algorithms are tested.

Scenario 2 – Autonomous Drone Software

A defense contractor is building software for autonomous drones. Safety, security, and regulatory compliance are critical. The system is large, complex, and requires rigorous risk analysis and iterative testing. Each phase must include detailed documentation and verification. Changes later in the project will be very expensive.

Scenario 3 – Social Media Startup

A small team is developing a new social media app with novel features like ephemeral posts, AI-based recommendations, and in-app games. Requirements are mostly vague, and the team expects frequent changes based on user testing and engagement data. The goal is to release updates frequently and rapidly.

Scenario 4 – E-Commerce Platform with Seasonal Deadlines

A retail company wants an e-commerce platform ready before a major holiday season. Core catalog, checkout, and payment modules are clearly defined. Marketing features like recommendations, promotions, and analytics may be added later. The team must release working modules quickly, with tight deadlines and incremental updates.